```
// Name: Scott Brandon Finkelstein
// Course: CS 201 - Data Structures and Algorithms
// Description: This program allows a user to search for values within a
string of text
// that contains the names of all fifty states.
package SearchTxt;

import java.util.Scanner;

public class Main {


    static int NO_OF_CHARS = 256;

    // The preprocessing function for Boyer Moore's
    // bad character heuristic
    static void badCharHeuristic(char[] str, int size, int badchar[]) {

        // Initialize all occurrences as -1
        for (int i = 0; i < NO_OF_CHARS; i++)
            badchar[i] = -1;

        // Fill the actual value of last occurrence
        // of a character (indices of table are ascii and
        // values are index of occurrence)
        for (int i = 0; i < size; i++)
            badchar[(int) str[i]] = i;
    }


    // A method that uses the Bad Character Heuristic of the BM algorithm
to search for a pattern
    static void search(char txt[], char pat[]) {
        int m = pat.length;
        int n = txt.length;

        int badchar[] = new int[NO_OF_CHARS];

        // Populate the bad character array by calling the preprocessing
function
        // badCharHeuristic() for the given pattern
        badCharHeuristic(pat, m, badchar);

        int s = 0; // s is shift of the pattern with respect to the text
        while (s <= (n - m)) {
            int j = m - 1;

            // Keep reducing index j of pattern while characters of
pattern and text are matching
            while (j >= 0 && pat[j] == txt[s + j])
                j--;

            // If the pattern is present at current shift, then index j
will become -1 after the
```

```
            // above loop
            if (j < 0) {
                System.out.println("Patterns occur at shift = " + s);

                // Shift the pattern so that the next character in text
aligns with the last
                // occurrence of it in pattern.
                s += (s + m < n) ? m - badchar[txt[s + m]] : 1;
            }

            else
                // shift the pattern so that the bad character in text
aligns with the last
                // occurrence of it in pattern.
                s += Math.max(1, j - badchar[txt[s + j]]);
        }
    }

    // prints the menu of options for the user to interact with
    private static void printMenu() {
        System.out.println("\n=== SEARCH MENU ===");
        System.out.println("1) Display the text");
        System.out.println("2) Search");
        System.out.println("3) Exit the program");
        System.out.println("====================");
    }

    public static void main(String[] args) {
        // the primary string of text that will be indexed with the
program
        String states =
                "Alabama Alaska Arizona Arkansas California Colorado
Connecticut Delaware Florida Georgia Hawaii Idaho Illinois Indiana Iowa
Kansas Kentucky Louisiana Maine Maryland Massachusetts Michigan Minnesota
Mississippi Missouri Montana Nebraska Nevada New Hampshire New Jersey New
Mexico New York North Carolina North Dakota Ohio Oklahoma Oregon
Pennsylvania Rhode Island South Carolina South Dakota Tennessee Texas
Utah Vermont Virginia Washington West Virginia Wisconsin Wyoming";
        // opens scanner to accept user input
        Scanner scanner = new Scanner(System.in);
        // program loops while running = true
        boolean running = true;

        while (running) {
            printMenu(); // show menu options 1-7
            int userInput = scanner.nextInt();
            // Displays the text in the states string
            if (userInput == 1) {
                System.out.println(states);
            } else if (userInput == 2) { // queries user for a pattern to
search for
                System.out.print("Enter the pattern you want to search
for: ");
                String pat = scanner.next();
```

```
                //
                search(states.toCharArray(), pat.toCharArray());
            } else if (userInput == 3) { // exits program
                System.out.println("Exiting program...");
                running = false;
            } else { // handling bad user inputs (!= 1, 2 or 3)
                System.out.println("Invalid input! Please enter 1, 2, or
3.");
            }
        }
        // close scanner to prevent resource leak
        scanner.close();
    }
}

// sources used:
// https://www.geeksforgeeks.org/dsa/boyer-moore-algorithm-for-pattern-
searching/
// https://study.com/academy/lesson/text-as-a-data-structure-java-
strings-character-arrays.html
// https://study.com/academy/lesson/string-searching-algorithms-methods-
types.html
```